Phase 2 Decision Record — Provider-Agnostic LLM Dispatch

Status

Planned for Phase 2, after the task-scoped agent protocol is functional and stable.

Context

AAAAT's first production-ready boundary is the task protocol:

- AAAAT owns private data, SQLite storage, rendering, validation, and review/apply logic.
- LLMs receive only task-scoped packets.
- LLMs return task results.
- AAAAT stores provenance and applies results deterministically only through review/apply flows.

This keeps the system provider-agnostic and avoids exposing broad private candidature data to agents.

However, a purely passive queue can feel unnatural to users. In the current model, the user creates or sees a task in AAAAT, then must open an external LLM tool and point it back to AAAAT. That is correct architecturally, but not ideal ergonomically.

Phase 2 should add a way for AAAAT to actively dispatch a task packet to an available LLM runner while preserving the same privacy boundary.

Core Conclusion

Do not try to "alert an LLM" directly. A bare LLM is not addressable.

Instead, AAAAT should dispatch a complete task packet to a configured runner.

A runner may be:

- a local model server;
- an installed LLM CLI;
- a user-defined script;
- an MCP host that supports sampling;
- a manual/outbox fallback.

The task packet remains the invariant. Dispatch backends are interchangeable adapters.

Target Model

```
AAAAT task queue
  -> task packet builder
  -> dispatch adapter
      -> manual/outbox
      -> command or installed CLI
      -> arbitrary user script
      -> local HTTP model server
      -> MCP sampling, if available
  -> result captured
  -> submit_agent_task_result
  -> user review/apply
```

Task Packet

A task packet should contain everything the LLM runner needs, and nothing more.

Minimum packet fields:

```
{
  "task_id": "...",
  "task_type": "...",
  "title": "...",
  "instructions": "...",
  "privacy_rules": "...",
  "context": {},
  "expected_output": {
    "format": "json_or_text",
    "schema": {}
  },
  "allowed_actions": [
    "submit_result"
  ],
  "callback": {
    "mode": "stdout_or_file_or_http",
    "task_id": "..."
  }
}
```

The packet must not contain:

- all candidatures;
- dashboard payloads;

- arbitrary search results;
- raw variables unless explicitly allowed by exposure policy;
- raw profile facts unless explicitly allowed by exposure policy;
- unrelated artifacts;
- unrelated text blobs;
- generic CRUD routes;
- direct database paths.

Dispatch Backends

1. Manual / Outbox Backend

Writes the packet to a local file or clipboard-friendly output.

Purpose:

- universal fallback;
- works with ChatGPT, Claude, Gemini, or any web UI;
- no provider integration required.

Example:

```
python -m aaaat.cli agent dispatch task_123 --backend manual
```

Potential output:

```
.private/agent_outbox/task_123.packet.json
```

2. Command Backend

Pipes the task packet to an installed command.

Purpose:

- supports Claude Code, Codex, OpenCode, Aider-like tools, local wrappers, or any future CLI;
- avoids provider SDKs;
- keeps AAAAT provider-agnostic.

Example:

```
python -m aaaat.cli agent dispatch task_123
  --backend command
  --cmd "claude -p"
```

The command backend should:

- send the packet through stdin or a temporary file;
- capture stdout as the candidate result;
- store stderr/logs as provenance or diagnostics;
- never auto-apply the result;
- fail safely if the command exits non-zero.

3. User Script Backend

Allows the user to provide an arbitrary local script as a compatibility bridge.

Purpose:

- maximizes open-source extensibility;
- lets users integrate unsupported providers without AAAAT core changes;
- supports custom local workflows, private wrappers, or organization-specific tooling.

Example:

```
python -m aaaat.cli agent dispatch task_123
  --backend script
  --script .private/dispatch/my_runner.py
```

Contract:

- AAAAT provides the task packet as stdin or as a file path argument.
- The script returns a result through stdout or a configured output file.
- AAAAT validates and stores the result as a task result.
- The script is user-owned and explicitly trusted by the user.
- Scripts are never stored in public examples with private credentials.

This backend is intentionally a controlled escape hatch for compatibility.

4. Local HTTP Backend

Sends the task packet to a local inference server.

Targets:

- Ollama;
- LM Studio;
- llama.cpp server;
- OpenAI-compatible local endpoints;
- other local inference servers.

Example:

```
python -m aaaat.cli agent dispatch task_123
  --backend local-http
  --base-url http://127.0.0.1:11434
  --model llama3.1:8b
```

Rules:

- no cloud API keys in core;
- no provider SDKs;
- local URL must be explicit;
- result is stored as task output with backend/model provenance;
- no auto-apply.

5. MCP Sampling Backend

Optional later adapter.

Purpose:

- allows AAAAT to request LLM sampling from a connected MCP host that supports sampling;
- useful when the user's active AI host already manages model access and permissions.

Limitations:

- not universal;
- requires a live MCP host/client relationship;
- should not be the first implementation;
- must still use the same task packet and task result path.

Dashboard Experience

The desired user experience is:

```
User clicks "Infer missing fields" in AAAAT
AAAAT creates or selects a task
AAAAT builds a minimal task packet
AAAAT dispatches it to the configured backend
The runner returns a result
AAAAT stores the result with provenance
Dashboard shows the result for review/apply
```

The user should not need to understand whether the runner is Ollama, Claude Code, OpenCode, llama.cpp, a custom script, or a manual packet.

The dashboard should show simple states:

- queued;
- dispatched;
- running, if trackable;
- result received;
- failed;
- reviewed;
- applied.

CLI Shape

Suggested commands:

```
python -m aaaat.cli agent dispatch <task_id> --backend manual

python -m aaaat.cli agent dispatch <task_id>
  --backend command
  --cmd "claude -p"

python -m aaaat.cli agent dispatch <task_id>
  --backend script
  --script .private/dispatch/my_runner.py

python -m aaaat.cli agent dispatch <task_id>
  --backend local-http
  --base-url http://127.0.0.1:11434
  --model llama3.1:8b

python -m aaaat.cli agent dispatch-next --backend command --cmd "opencode"
```

The existing task protocol commands remain:

```
python -m aaaat.cli agent tasks
python -m aaaat.cli agent context <task_id>
python -m aaaat.cli agent submit <task_id> --result-file result.json
python -m aaaat.cli agent claim <task_id>
python -m aaaat.cli agent release <task_id>
```

Configuration

Dispatch configuration should be private and local.

Suggested private config path:

```
.private/dispatch.json
```

Example:

```
{
  "default_backend": "command",
  "backends": {
    "claude_local": {
      "type": "command",
      "cmd": "claude -p"
    },
    "ollama_default": {
      "type": "local-http",
      "base_url": "http://127.0.0.1:11434",
      "model": "llama3.1:8b"
    },
    "custom_runner": {
      "type": "script",
      "script": ".private/dispatch/my_runner.py"
    }
  }
}
```

No provider credentials should be required by AAAAT core. If a user script or external CLI uses credentials, that remains outside AAAAT's core configuration.

Privacy and Safety Rules

Dispatch does not weaken the privacy model.

Rules:

- dispatch only receives a task packet;
- no backend receives the full database;
- no backend receives dashboard payloads;
- no backend gets arbitrary search tools;
- no backend directly mutates candidature/profile data;
- results are stored with provenance;
- deterministic review/apply remains the mutation point;
- auto-apply is not part of Phase 2.

If the user grants a script, command, or external agent arbitrary filesystem or shell access, that is outside AAAAT's enforceable privacy boundary. AAAAT should document this clearly.

Suggested Module Layout

```
aaaat/
  agent_access.py
  dispatch/
    __init__.py
    packet.py
    base.py
    manual.py
    command.py
    script.py
    local_http.py
    runner.py
```

Optional later:

```
    mcp_sampling.py
```

Phase 2 Acceptance Criteria

Phase 2 is complete when:

- AAAAT can build a complete task packet from a task id.
- AAAAT can dispatch the packet through at least one backend.
- Manual/outbox backend works.
- Command or script backend works.
- Returned output is stored through `submit_agent_task_result`.
- Result provenance includes backend type, command/script/model where applicable, timestamps, and diagnostics.
- Dashboard can trigger dispatch for a selected task without exposing broad private JSON APIs.
- No result is auto-applied.
- No provider SDK or cloud dependency is added.
- User-defined scripts are supported as a compatibility escape hatch.
- Documentation explains that dispatch adapters are replaceable and that the task protocol remains the core boundary.

Non-Goals

Phase 2 should not implement:

- autonomous multi-step tool loops;
- direct database access for LLMs;
- broad search tools for LLMs;
- provider-specific paid API onboarding;

- cloud account management;
- API key storage in public config;
- auto-apply;
- Electron/Tauri/native UI;
- a full MCP runtime if simple dispatch backends are not finished first.

Final Decision

AAAAT should evolve from a passive task queue into a provider-agnostic task dispatcher.

The correct abstraction is not provider integration. The correct abstraction is:

```
task packet -> replaceable dispatch backend -> task result
```

This preserves the privacy model, keeps the app local-first, and gives users a natural dashboard experience without binding AAAAT to any single LLM provider or runtime.